

1. Introducción

El objetivo de este proyecto consiste en implementar físicamente en PostgreSQL una base de datos espacial diseñada previamente a nivel conceptual y lógico. Para ello se ha utilizado la extensión PostGIS, que permite trabajar con información geográfica dentro de un sistema gestor de bases de datos relacional.

La base de datos desarrollada permite almacenar información sobre zonas naturales, rutas, puntos de interés asociados a las rutas y reseñas realizadas por los usuarios. Además, se ha integrado información geográfica mediante la importación de una capa espacial utilizando herramientas de software libre.

Esta implementación incluye la creación de la estructura de tablas, la definición de claves primarias y externas para garantizar la integridad referencial, la importación de datos desde archivos CSV y la creación de índices que optimizan el rendimiento de las consultas.

2. Software y herramientas utilizadas

Para la realización de esta práctica se han utilizado las siguientes herramientas:

- **PostgreSQL** como sistema gestor de bases de datos relacional.
- **PostGIS** como extensión espacial que permite almacenar y analizar datos geográficos.
- **pgAdmin** como interfaz gráfica para la gestión de la base de datos.
- **OGR2OGR** para la importación de datos espaciales desde archivos geográficos.
- **OSGeo4W Shell** para ejecutar los comandos de importación espacial.

El uso combinado de estas herramientas permite gestionar bases de datos espaciales de forma eficiente y reproducible, integrando información geográfica y datos alfanuméricos dentro de un mismo sistema.

3. Activación de la extensión espacial

El primer paso para trabajar con datos espaciales en PostgreSQL consiste en activar la extensión PostGIS dentro de la base de datos. Esta extensión añade nuevos tipos de datos, funciones y operadores que permiten almacenar y manipular información geográfica.

La activación de la extensión se realizó mediante el siguiente comando SQL:

```
CREATE EXTENSION postgis;
```

Una vez activada la extensión, es posible utilizar el tipo de dato **geometry**, que permite almacenar geometrías como puntos, líneas o polígonos.

Asimismo, PostGIS incorpora numerosas funciones espaciales que permiten llevar a cabo análisis geográficos directamente dentro de la base de datos, como cálculos de distancia, intersecciones o generación de nuevas geometrías.

4. Importación de datos espaciales

La información geográfica correspondiente a las zonas naturales se encontraba almacenada en un archivo geoespacial en formato **GeoPackage (.gpkg)**. Para importar esta información en PostgreSQL se utilizó la herramienta **ogr2ogr**, que forma parte de la biblioteca GDAL y permite convertir datos geográficos entre distintos formatos.

El comando utilizado fue el siguiente:

```
-- (ejecutar en OSGeo4W Shell, no en pgAdmin)

-- ogr2ogr -f "PostgreSQL" "PG:host=localhost port=5432 dbname=practica_bd user=postgres"
^

-- "C:\Users\ronaldalvarez\Downloads\zonas_naturales\zonas_naturales\zonas_naturales.gpkg" ^

-- -nln public.zonas_naturales -overwrite ^

-- -lco GEOMETRY_NAME=geom -lco FID=gid ^

-- -nlt PROMOTE_TO_MULTI
```

Este proceso generó automáticamente la tabla **zonas_naturales** dentro de PostgreSQL, incluyendo la columna **geom** que almacena la geometría de cada entidad espacial, así como un identificador único denominado **gid**. La geometría se almacena como tipo **geometry**, permitiendo realizar consultas espaciales.

5. Creación de la estructura de tablas

A partir del modelo lógico diseñado previamente, se procedió a la creación de las tablas necesarias para almacenar la información de la aplicación.

Las principales tablas creadas fueron:

- **usuario**, que almacena la información de los usuarios del sistema.
- **ruta**, que contiene las rutas disponibles en las distintas zonas naturales.
- **punto_ruta**, que almacena los puntos de interés asociados a cada ruta.
- **resena**, que recoge las valoraciones y comentarios de los usuarios.
- **ruta_tipo**, que permite clasificar las rutas según diferentes tipos.

Las tablas fueron creadas mediante comandos SQL de definición de datos (**CREATE TABLE**), definiendo claves primarias para garantizar la identificación única de cada registro dentro de la base de datos.

6. Definición de claves externas

Una vez creadas las tablas, se establecieron las relaciones entre ellas mediante la definición de claves externas. Estas restricciones permiten mantener la integridad referencial entre las distintas entidades de la base de datos.

Las principales relaciones definidas fueron:

- La tabla **ruta** se relaciona con **zonas_naturales**, indicando en qué **zona natural** se encuentra cada ruta.
- La tabla **punto_ruta** se relaciona con **ruta**, ya que cada punto pertenece a una ruta concreta.
- La tabla **reseña (resena)** se relaciona tanto con **usuario** como con **ruta**, indicando qué usuario ha realizado la reseña y sobre qué ruta.

La definición de estas relaciones permite evitar inconsistencias en los datos y asegura que todas las referencias entre tablas sean válidas.

7. Importación de datos desde archivos CSV

Una vez creada la estructura de la base de datos, se procedió a la carga de datos mediante el comando **COPY**, que permite importar grandes volúmenes de información desde archivos externos en formato CSV.

Por ejemplo, para importar los datos de usuarios se utilizó el comando:

```
COPY usuario FROM 'usuario.csv' WITH (FORMAT csv, HEADER true);
```

Los archivos utilizados fueron:

- **usuario.csv**
- **ruta.csv**
- **punto_ruta.csv**
- **resena.csv**

El uso del comando **COPY** permite realizar la carga de datos de forma eficiente directamente desde el sistema de archivos hacia las tablas de la base de datos.

En el caso de la tabla **punto_ruta**, los archivos CSV contenían las coordenadas geográficas separadas en columnas de longitud y latitud. Por esta razón, fue necesario crear la geometría después usando la función espacial **ST_MakePoint**, asignando también el sistema de referencia espacial **WGS84 (SRID 4326)**.

8. Creación de índices adicionales

Para mejorar el rendimiento de las consultas, se crearon índices adicionales en aquellas columnas utilizadas con mayor frecuencia en operaciones de unión entre tablas.

Se añadieron índices sobre las claves externas de las tablas **ruta**, **punto_ruta** y **resena**, lo que permite optimizar las consultas que relacionan estas tablas.

Además, se crearon índices espaciales de tipo **GiST** sobre las columnas geométricas. Estos índices permiten acelerar significativamente las consultas espaciales, como por ejemplo la búsqueda de elementos dentro de una determinada zona geográfica o el cálculo de proximidades.

Implementación física de una base de datos espacial en PostgreSQL/PostGIS

En el caso de las columnas geométricas, se emplearon índices espaciales de tipo **GiST**, que son los más utilizados en PostgreSQL para optimizar consultas espaciales.

9. Verificación de la carga de datos

Para comprobar que la importación de datos se realizó correctamente, se ejecutaron diversas consultas de verificación, como el recuento de registros en cada tabla.

Estas comprobaciones permitieron confirmar que los datos habían sido cargados correctamente en la base de datos y que las relaciones entre las distintas tablas funcionaban adecuadamente.

Por ejemplo, se utilizaron consultas como `SELECT COUNT(*) FROM ruta;` para comprobar que los registros se importaron correctamente.

Para comprobar que la carga de datos se realizó correctamente, se ejecutaron consultas de verificación que permiten contar el número de registros presentes en cada tabla. Por ejemplo, se utilizaron consultas como las siguientes:

```
SELECT COUNT(*) FROM usuario;
```

```
SELECT COUNT(*) FROM ruta;
```

```
SELECT COUNT(*) FROM punto_ruta;
```

```
SELECT COUNT(*) FROM resena;
```

```
SELECT COUNT(*) FROM zonas_naturales;
```

Además, se realizaron consultas de prueba que combinan varias tablas mediante operaciones de unión (JOIN) para verificar que las relaciones entre entidades funcionan correctamente.

10. Conclusiones

La implementación física de la base de datos permitió trasladar el modelo conceptual y lógico previamente diseñado a un entorno real de gestión de datos espaciales.

El uso de PostgreSQL junto con la extensión PostGIS proporciona una plataforma potente y flexible para el almacenamiento y análisis de información geográfica. Asimismo, la utilización de herramientas como ogr2ogr facilita la integración de datos espaciales procedentes de diferentes formatos.

Finalmente, la correcta definición de claves primarias, claves externas e índices permite garantizar la integridad de los datos y optimizar el rendimiento del sistema, lo que resulta fundamental en aplicaciones que manejan información geográfica.

En conjunto, la práctica permite comprender el proceso completo de implementación de una base de datos espacial, desde el diseño del modelo hasta la carga y gestión de los datos dentro de PostgreSQL/PostGIS.

Como resultado, se obtuvo una base de datos espacial operativa, capaz de almacenar zonas naturales, rutas, puntos de interés y reseñas, manteniendo relaciones consistentes entre las tablas.